

Machine-Level Data Representation

Binary/Hex representation, signed vs unsigned, two's complement, and memory endianness / alignment in AArch64

Computer Architecture & Organization | Topic 3

Topic 3 Technical Roadmap

Topic Objective

Examine how machine-level information is encoded, interpreted, and organized across fixed-width registers and byte-addressed memory structures.

- **Section 1:** Bits, Bytes, Words, and Base Conversions
- **Section 2:** Binary and Hexadecimal Representation
- **Section 3:** Unsigned and Signed Integer Representations
- **Section 4:** Two's Complement Encoding and Arithmetic Behavior
- **Section 5:** Memory Endianness and Data Alignment in AArch64
- **Section 6:** Topic Summary and Worked Self-Test Exercises

Section 01

Bits, Bytes, Words, and Base Conversions

Why Data Representation Matters

All machine-level information is stored as fixed-width bit patterns:

- **Context Dependent:** The same bit pattern can mean different things under different interpretations (e.g., 0x0xFF can be 255 unsigned or -1 signed).
- **ISA Rules:** Software interpretation determines whether bits represent data items, memory addresses, or machine instructions.
- **Fixed-Width Bounds:** Registers impose strict hardware width limits on numeric ranges.
- **Layout Rules:** Memory organization dictates how multi-byte values are mapped and retrieved.

Data Units in Memory and Registers

Hierarchical Units of Storage

- **Bit:** A single binary digit (0 or 1).
- **Byte:** Contains 8 bits; the foundational unit of addressable memory.
- **Halfword:** Commonly 16 bits (2 bytes).
- **Word:** Commonly 32 bits (4 bytes) in standard architectures.
- **Doubleword:** Commonly 64 bits (8 bytes), native to AArch64 registers (x0–x30).

Example

One AArch64 register width = 64 bits = 8 bytes.

The Binary Number System (Base 2)

Binary maps directly to physical dual-state digital hardware:

- Each digit position represents a distinct power of two.
- **LSB (Least Significant Bit)**: The rightmost bit position (2^0).
- **MSB (Most Significant Bit)**: The leftmost bit position in a fixed-width container.

Positional Evaluation Example

$$1011_2 = (1 \times 8) + (0 \times 4) + (1 \times 2) + (1 \times 1) = 8 + 0 + 2 + 1 = 11_{10}$$

Base Conversion Mechanics

Binary to Decimal

- Write power-of-two weights above each bit ($2^0, 2^1, 2^2 \dots$).
- Disregard positions where the bit is 0.
- Sum the weights where the bit is 1.
- Example:
 $01010110_2 = 64 + 16 + 4 + 2 = 86_{10}$.

Decimal to Binary

- Repeatedly divide the decimal value by 2.
- Record each remainder as a binary digit.
- Read remainders from bottom to top.
- Pad with leading zeros to match container width.
- Example: $13_{10} = 1101_2$.

Binary and Hexadecimal Representation

The Hexadecimal Number System (Base 16)

Hexadecimal provides a compact, readable shorthand for machine-level data:

- Digits range from 0 to 9 and letters *A* to *F* (representing values 10 to 15).
- **Exact Mapping:** Each single hexadecimal digit maps precisely to 4 binary bits ($2^4 = 16$).
- Used universally for memory addresses, instruction opcodes, and register dumps.

Example

$$0x2A = 0010\ 1010_2 = (2 \times 16) + 10 = 42_{10}$$

Direct Binary-Hexadecimal Conversion

Converting between binary and hex requires no intermediate decimal math:

Binary to Hexadecimal

- Group binary bits into sets of four starting from the right (LSB).
- Pad the leftmost group with leading zeros if necessary.
- Map each 4-bit nibble directly to its hex digit.
- Example: $1101\,0110_2 = 0xD6$.

Hexadecimal to Binary

- Take each individual hex character.
- Expand it into its exact 4-bit binary equivalent.
- Concatenate the nibbles together.
- Example: $0x5F = 0101\,1111_2$.

AArch64 Registers and Data Widths

AArch64 general-purpose registers are 64 bits wide, supporting flexible data sizing:

- A 64-bit register (`x0–x30`) holds 64 bits of raw data bits.
- The exact same bit pattern can be interpreted as signed integers, unsigned integers, pointers, or masks.
- Memory instructions (`ldr`, `str`) explicitly specify access sizes (bytes, halfwords, words, doublewords).

Example Interpretation

Setting `x0 = 0xFFFFFFFFFFFFFFFF` can represent the unsigned integer 18,446,744,073,709,551,615 or the signed integer -1 .

Unsigned and Signed Integer Representations

Unsigned Integer Representation

Unsigned integers represent only non-negative values (0 and positive integers):

- Every bit position contributes a strictly positive power-of-two weight.
- The leftmost bit (MSB) acts as a high-order magnitude bit, not a sign indicator.
- An n -bit unsigned container spans the exact range from 0 to $2^n - 1$.

Unsigned Range Examples

- 8-bit unsigned range: 0 to $2^8 - 1 = 255$.
- 64-bit unsigned range: 0 to $2^{64} - 1 \approx 1.84 \times 10^{19}$.
- Example: $1111\ 1111_2 = 255$ (in 8 bits).

Unsigned Arithmetic and Wraparound

Unsigned operations are evaluated modulo 2^n for an n -bit container:

- **Overflow Behavior:** If an arithmetic operation exceeds the maximum representable value ($2^n - 1$), high-order carry bits are discarded.
- **Wraparound:** The stored result wraps around circularly within the fixed container width.
- Processors record unsigned carry-out conditions in status flags.

Classic Wraparound Example

In 8-bit unsigned arithmetic:

$$255 + 1 = 1111\ 1111_2 + 1 = 0000\ 0000_2 = 0$$

Signed Integer Challenges & Sign-Magnitude Failure

Signed integers require encoding both positive and negative values:

- **Naïve Approach (Sign-Magnitude):** Dedicate the MSB as a sign flag (0 = positive, 1 = negative).
- **Fatal Flaws of Sign-Magnitude:**
 - **Dual Zeros:** Encodes both positive zero (00000) and negative zero (10000).
 - **Broken Arithmetic:** Standard hardware adders fail when adding positive and negative numbers (e.g., $+3 + (-3) \rightarrow 00011 + 10011 = 10110(-6)$, which is incorrect).

The Engineering Solution

Modern computer systems bypass these hardware flaws by adopting **two's complement representation**.

Two's Complement Representation and Arithmetic Behavior

Two's Complement Representation

Core Design Objectives

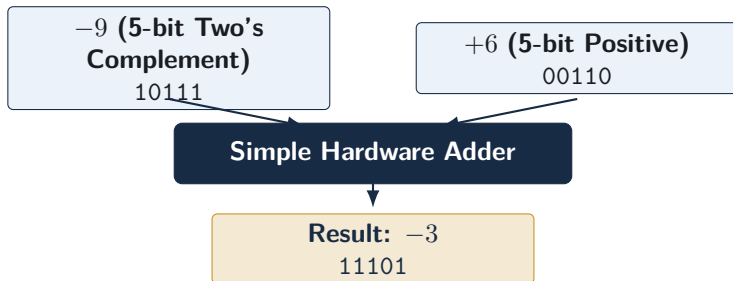
Two's complement provides a robust signed number system requiring **no special additional hardware** for add, subtract, or multiply units, while maintaining a single unique zero.

Algorithm for Negation (Forming Two's Complement)

- 1 Start with the positive magnitude in binary.
 - 2 **Invert every bit** ($0 \rightarrow 1, 1 \rightarrow 0$) to produce the one's complement.
 - 3 **Add 1** to the inverted result to produce the final two's complement encoding.
- Example (-5 in 8 bits): $+5 = 0000\ 0101 \rightarrow \text{invert} = 1111\ 1010 \rightarrow \text{add } 1 = 1111\ 1011$.

Two's Complement in Action: $-9 + 6$

Standard hardware adders evaluate signed numbers correctly without modification:



- Adding 10111 (-9) and 00110 ($+6$) yields 11101 (which evaluates exactly to -3).

Two's Complement Numeric Range

An n -bit container under two's complement encoding spans asymmetric bounds:

- **Total Patterns:** 2^n unique bit combinations.
- **Signed Range:** -2^{n-1} to $2^{n-1} - 1$.
- **Asymmetry:** There is always **one more negative value** than positive value because zero consumes one positive pattern.
- The most negative value has only its sign bit set ($1000 \dots 0$).

8-Bit and 64-Bit Ranges

- 8-bit two's complement range: -128 to $+127$.
- 64-bit two's complement range: $-9,223,372,036,854,775,808$ to $+9,223,372,036,854,775,807$.

Signed Arithmetic Overflow

Definition of Signed Overflow

Signed overflow occurs when the true mathematical result of an operation falls outside the representable signed range of the destination container.

- **Symptom:** Adding two positive numbers yields a negative result, or adding two negative numbers yields a positive result.
- **Detection:** Hardware cannot detect overflow merely by looking at the final bit pattern; it evaluates carry inputs and outputs into the MSB.
- **Status Flags:** AArch32/AArch64 condition flags record signed overflow in the **V** (Overflow) flag.
- **Example:** In 8-bit math, $127 + 1 = 0111\ 1111 + 0000\ 0001 = 1000\ 0000(-128)$, triggering an immediate overflow exception/flag.

Memory Endianness and Data Alignment in AArch64

Memory as a Byte-Addressed Structure

AArch64 memory is organized as a linear array of individually addressable bytes:

- Each unique memory address references exactly one 8-bit byte.
- Multi-byte data structures (halfwords, words, doublewords) occupy consecutive sequential byte addresses.
- **Address Pointer:** A 64-bit multi-byte value is identified by the address of its lowest-numbered byte container.

Example: 64-Bit Doubleword Layout

A 64-bit doubleword stored at pointer address 0x0x1000 occupies physical byte addresses 0x0x1000, 0x0x1001, 0x0x1002, 0x0x1003, 0x0x1004, 0x0x1005, 0x0x1006, and 0x0x1007.

Endianness: Ordering Multi-Byte Values

Definition

Endianness defines the sequential byte ordering used when storing multi-byte values across consecutive memory addresses.

Big-Endian Ordering

- Stores the **most significant byte (MSB)** at the lowest memory address.
- Matches reading human numerical text (left-to-right).

Little-Endian Ordering

- Stores the **least significant byte (LSB)** at the lowest memory address.
- Standard default layout for modern ARM64 and x86 architectures.

Little-Endian Layout in AArch64 Practice

AArch64 systems default to little-endian memory organization:

- The underlying numeric value held in a register remains unaffected by byte layout.
- Load instructions reconstruct registers from memory bytes; store instructions decompose register values into byte streams.
- Endianness becomes explicitly visible when inspecting raw memory byte dumps.

Concrete Layout Example

Storing 64-bit value `0x0x1234567890ABCDEF` at address `0x0x2000` in little-endian order places bytes in ascending memory addresses as:

`0xEF` (at `0x2000`), `0xCD`, `0xAB`, `0x90`, `0x78`, `0x56`, `0x34`, `0x12` (at `0x2007`)

Data Alignment in AArch64

Definition of Natural Alignment

Data alignment requires that a variable of size n bytes be stored at a memory address that is an exact multiple of n .

- **64-Bit Doubleword:** Naturally aligned at addresses divisible by 8 (0, 8, 16 ...).
- **32-Bit Word:** Naturally aligned at addresses divisible by 4.
- **16-Bit Halfword:** Naturally aligned at addresses divisible by 2.
- **Hardware Efficiency:** Aligned memory transactions execute with maximum speed and single-cycle bus efficiency.
- Misaligned accesses can trigger hardware traps, alignment faults, or severe performance penalties.

Summary & Self-Test

Topic 3 Comprehensive Summary

- **Fixed-Width Representation:** Information is stored as bit patterns whose semantics depend entirely on software and ISA interpretation.
- **Base Conversions:** Binary (*Base* 2) and Hexadecimal (*Base* 16) provide readable notations for digital machine states.
- **Unsigned Integers:** Represent non-negative values with strict wraparound modulo 2^n .
- **Two's Complement:** The industry standard for signed integers, enabling negative values without special ALU hardware.
- **Endianness & Alignment:** AArch64 utilizes little-endian byte ordering and strict natural data alignment for memory access performance.

Self-Test: Questions 1 & 2

Question 1: Bits, Words, and Bases

What is the relationship between bits, bytes, words, and 64-bit AArch64 registers? Why is hexadecimal universally preferred over binary when inspecting memory dumps?

Question 2: Unsigned Wraparound

What range of values can be represented by an unsigned n -bit integer? What happens when an 8-bit unsigned accumulator holding 255 has 1 added to it?

Self-Test: Solutions 1 & 2

Solution 1

Bits are binary digits; 8 bits make a byte; words are 32 bits; AArch64 general-purpose registers are 64 bits (doublewords) wide. Hexadecimal is compact, mapping each single character directly to a 4-bit nibble without conversion fuzziness.

Solution 2

An unsigned n -bit container spans 0 to $2^n - 1$. Adding 1 to 255 in an 8-bit container triggers arithmetic wraparound modulo 2^8 , resulting in 0 and setting the carry flag.

Self-Test: Questions 3 & 4

Question 3: Two's Complement Negation

How do you form the two's complement negative of a positive binary number? What signed range is representable in a 32-bit two's complement integer container?

Question 4: Signed vs. Unsigned Interpretation

How can the exact same 32-bit bit pattern represent two completely different values when interpreted as signed versus unsigned?

Self-Test: Solutions 3 & 4

Solution 3

To form a two's complement negative, take the binary magnitude, invert every bit (one's complement), and add 1. A 32-bit two's complement container spans -2^{31} to $2^{31} - 1$ ($-2,147,483,648$ to $+2,147,483,647$).

Solution 4

Unsigned interpretation treats all bits as positive power-of-two weights. Signed two's complement interpretation assigns the MSB a negative weight. Thus, `0x0xFFFFFFFF` equals $4,294,967,295$ unsigned but -1 signed.

Self-Test: Questions 5 & 6

Question 5: Endianness in Memory

What is the fundamental difference between little-endian and big-endian byte ordering?
Which convention does AArch64 use by default?

Question 6: Data Alignment

What does data alignment mean in computer architecture, and why are naturally aligned memory addresses important for performance?

Self-Test: Solutions 5 & 6

Solution 5

Little-endian stores the least significant byte (LSB) at the lowest memory address, whereas big-endian stores the most significant byte (MSB) at the lowest address. AArch64 uses little-endian byte ordering by default.

Solution 6

Data alignment requires variables of size n to be stored at addresses divisible by n . Natural alignment ensures hardware can fetch multi-byte operands efficiently in single memory bus cycles without alignment penalty delays.

Review: Summary Table of Data Types

Representation Type	Bit Interpretation	8-Bit Container Range
Unsigned Integer	All bits represent positive magnitudes	0 to 255
Signed Two's Complement	MSB has negative positional weight	-128 to +127
Hexadecimal Notation	4-bit nibble groupings for readability	0x0x00 to 0x0xFF
Little-Endian Memory	LSB stored at lowest byte address	Sequential byte mapping

Wrap-Up and Next Steps

Moving Forward to Topic 4

Now that we understand how integers, bit patterns, and data layouts are represented at the machine level, we will build upon this foundation to study:

- **Integer Arithmetic & Floating-Point Representation:** How ALUs execute shifts, multiplications, and IEEE-754 floating-point values.
- **Advanced AArch64 Addressing Modes:** Direct, indirect, register-offset, and stack pointer manipulation in assembly.

End of Topic 3 — Questions & Discussion